SwarmBox · 多智能体不可逃逸测试床

2026 华为开发者大赛 · 智能体创新赛（北部赛区） | 参赛主体：OPC（一人公司 / 超级个体） | 场景方向：企业场景 · AI 安全与流程创新

235	0	31/32	5–20	0
断言全部通过（0 失败）	逃逸成功次数	攻击语料命中预期规则	智能体规模可调	第三方依赖 · 零网络

一、场景背景

多智能体系统正从实验室走向生产。一旦给一群智能体授予工具与共享资源，问题就从"单个模型会不会乱说"升级为"**一群智能体会不会合谋、逃逸、耗尽资源**"——而这在上線前几乎无法系统性检验。

二、真实痛点（三条）

- **逃逸风险无法预检**：智能体被授予工具权限后，文件/网络/进程等逃逸路径难以穷举验证。
- **禁止清单式沙箱易绕过**：靠"不许调用 X"实现的隔离，总会有遗漏，且**无法证明**已穷尽。
- **群体行为缺实验环境**：合谋、资源耗尽等现象缺少可复现、可量化的观测平台。

目标用户：需要在上线前评估多智能体系统安全性的企业与超级个体；AI 对齐研究者。

三、SwarmBox 的根本做法

它不解释执行智能体的输出，而是把输出当作纯数据来解析。

因此文件、网络、进程、import、反射这些逃逸手段**根本不在语法里**——不是被禁止，而是**无从调用**。

四、实测（本机可复现，零网络）

命令	结果
python eval.py	`235 passed / 0 failed`
swarmbox gate	`32 语料 / 31 命中 / escape 0`
swarmbox run	`violations 0 · corr -0.831`
swarmbox fidelity	`collapse 1.0000 · gini 0.1649`

⚠️ 按实测值填报：README 旧文案"32 条逐条被拒"实为 **31/32 命中预期规则 ID**（escape successes 仍为 0）；fidelity 基尼实跑 **0.1649**。提交材料一律用实跑值。

功能架构：三重校验 + 只追加账本

提案 → 结构校验 → 身份校验 → 状态校验 → 闸门汇总 → 只追加账本 | 不可逃逸是可审计的，而非需被信任的

闸门链路（gate.py）

`prop` → `check_shape` → `check_identity` → `check_state` → `screen` → `Ledger`

模块	关键符号	职责
gate.py	`screen()` / `check_shape()` `check_identity()` / `check_state()`	三重校验 + 闸门汇总
ledger.py	`class Ledger` / `read_ledger()`	**只追加**账本，记录全部被拒提案
world.py	`class World` / `InvariantError`	公共资源池 + 不变量守护
probe.py	攻击语料注入	验证不可逃逸性（32 条）
fidelity.py	`aggregate()` / `compare()`	保真度与对照实验
metrics.py / swarm.py / agents.py / chain.py	—	指标、群体调度、智能体、链式实验

为何"不可逃逸"可审计

每一次被拒提案都写入**只追加账本**，事后可完整回放：谁、在第几轮、试图做什么、命中哪条规则。安全性质不是声称的，是**可核对的**。

真实运行输出

```
$ python -m swarmbox gate
32 corpus items
31 refused with expected rule
escape successes 0

$ python -m swarmbox run
[bounds: 5-20 agents]
invariants 0 violations
collusion corr -0.831
covert channel 10 cross-agent reads

$ python -m swarmbox fidelity
collapse_rate 1.0000
gini 0.1649
escape 0  violations 0
```

corr −0.831 的读法：强负相关意味着智能体之间存在明显的**此消彼长**关系——一方取用资源时另一方被压制，是资源竞争/合谋结构的量化证据，而非随机噪声。

全部命令零网络、零 API Key，评委 `git clone` 后可直接复现。

核心亮点：逃逸手段不在语法里

差异化创新点 · 对应评分维度「专业贴合与创新 25 分」

亮点 1 · 不解释执行，只作数据解析

常见沙箱的做法是"执行智能体输出，但禁止其中某些操作"——本质是**黑名单**，永远有遗漏。SwarmBox 反过来：智能体的输出被当作**纯数据结构**解析与校验，**从不作为代码执行**。因此文件、网络、进程、import、反射这些逃逸手段**根本不在语法里**。 这不是"禁止得更严"，而是"攻击面不存在"。

亮点 2 · 三重校验层层收窄

- **check_shape**：提案形态是否合法（结构层）
- **check_identity**：是否为该智能体本人（身份层）
- **check_state**：当前世界状态下是否允许（状态层）

任一不过即由 `screen()` 拒绝并记账，不进入世界状态。

亮点 3 · 可审计而非需信任

只追加账本使"从未发生逃逸"成为**可核对的历史记录**，而不是一句承诺。

亮点 4 · 群体行为可量化

- 5–20 个智能体共享公共资源池，多轮交互
- 可观测**协作 / 合谋 / 资源耗尽**三种群体形态
- collapse_rate、gini、collusion corr 等指标量化群体状态
- `compare()` 支持对照实验，便于做消融研究

亮点 5 · 零依赖 + 完全离线可复现

- 纯 Python 标准库，**pip install 无内容**，**零网络访问**
- Python 3.9+，`git clone && python eval.py` 一次跑通 235 条断言
- 附 REPORT.pdf 与 eval_chain.py 链式实验

落地价值

可直接作为企业多智能体系统**上线前的安全评估基线**；也可作为 AI 对齐研究中"合谋与资源耗尽"现象的标准实验环境。

华为云码道（CodeArts）应用说明

对应评分维度「码道深度使用 25 分」—— 覆盖环境搭建 / 代码生成 / 调试排错 / 文档生成 全流程

码道在开发全流程的覆盖

环节	码道承担的工作	产物
环境搭建	生成一键安装脚本与环境自检工具（本项目零依赖，脚本需说明并跳过安装）	scripts/setup.sh · doctor.py
代码生成	新增第四重校验 check_rate；补充 8 条新型逃逸语料	gate.py · probe.py · 新断言
调试排错	修复 Unicode 同形字伪装身份导致的身份校验不稳定	gate.py 修复 + 回归断言
文档生成	架构与复现指南、实验指标说明	readme/ 全套 Markdown

Prompt 设计要点

- 每条提示词**指明真实符号**（如 `screen(prop, world, agent_id)`、`class Ledger`），避免码道臆造接口。
- 显式约束：**纯标准库、阈值不得来自提案内容、不得引入可被智能体输出影响的参数**——保证改动不破坏核心安全性质。
- 要求码道**同时输出断言代码**，使每次生成都受 235 条回归测试约束。

使用证明材料（codearts_proof/）

- 核心功能开发的 **AI 交互对话与 Prompt 截图**（A/B/C/D 四组共 6 条）
- 代码生成、重构、排错的**过程记录**
- 码道 IDE 插件**操作日志与记录**
- 每条提示词对应的**本地终端执行输出**（含 eval.py / gate 结果）

交付规范（5 文件夹）

```
SwarmBox_<队名>_v1.0_<联系方式>.zip
├─ ppt/              作品介绍 PDF
├─ project/          可运行源码
├─ video/            60-90s 演示录屏
├─ codearts_proof/   码道使用证明
└─ readme/           说明文档
```

真实性声明：全部码道使用痕迹为真实开发过程记录，未做任何合成或伪造。
并行说明：中部（PayPilot）、西北（QuorumPilot）、北部（SwarmBox）为三个不同作品分投不同赛区，码道动作可在同一场会话连做。
待补：本单 video/ 需 60–90 秒演示录屏。